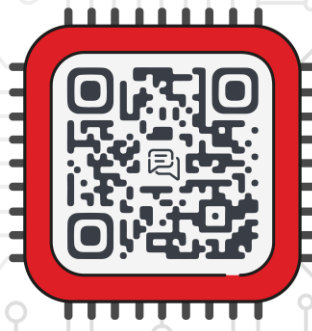




نظري

4

36

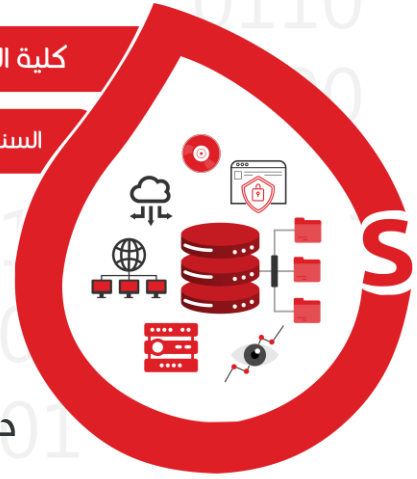
5400 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Indexing & Hashing

د. مادلين عبود



RB Informatix; 23/10/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

تُعد عملية الوصول السريع والفعال إلى البيانات من أهم التحديات في أنظمة قواعد البيانات، حيث لا يكفي مجرد تخزين السجلات بشكل متسلسل داخل الملفات. هنا يظهر دور **البنى الإضافية** **المساعدة للتخزين مثل الفهارس (Indexing) والتجزئة (Hashing)**، والتي صُممت خصيصاً لتسريع عمليات البحث والاستعلام وتقليل زمن الوصول إلى البيانات.

أقسام المحاضرة:

1. المفاهيم الأساسية Basic Concepts.
2. الأدلة المرتبة ordered indices.
3. ملفات الفهرسة من النوع B+-Tree Index Files - B+Tree.
4. ملفات الفهرسة من النوع B-Tree Index Files - B-Tree.
5. التقطيع الثابت Static Hashing.
6. المقارنة بين الفهرسة المرتبة والتقطيع Comparison of Ordered Indexing and Hashing.
7. تعريف الفهرس في SQL - SQL Index Definition.
8. الوصول من خلال مفاتيح متعددة Multiple-Key Access.

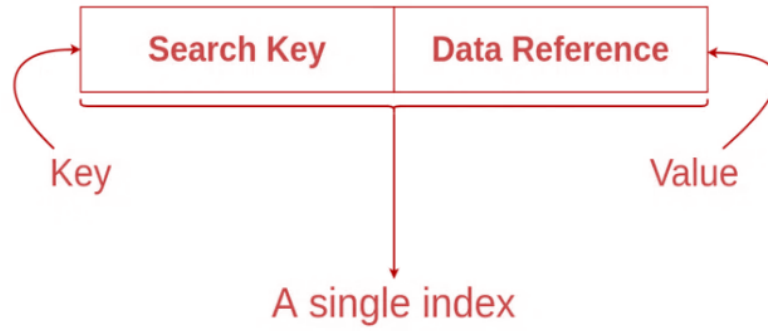
المفاهيم الأساسية

الفهرس (index): هو بنية بيانات تستخدم لتسريع الوصول إلى السجلات داخل ملف البيانات الأساسي، تماماً مثل فهرس الكتاب الذي يساعدنا في الوصول إلى الصفحة المطلوبة بسرعة دون قراءة الكتاب كاملاً.

مفتاح البحث (Search Key): هو واصفة أو مجموعة واصفات تستخدم للبحث عن تسجيلة، مثل رقم الحساب أو اسم الفرع.

ملف الفهرس (Index File): يتكون من مجموعة من التسجيلات تسمى مداخل الفهرس، وكل مدخل يحوي:

- قيمة مفتاح البحث.
- مؤشر (Pointer) يشير إلى موقع التسجيلة أو الكتلة التي تحتويها في الملف الأساسي.



ملف الفهرس يكون أصغر بكثير من الملف الأساسي، لأننا لا نخزن كل الحقول، بل فقط مفاتيح البحث والمؤشرات.

يوجد نوعان أساسيان من الفهارس:

- الأدلة المرتبة (Ordered Indices): مفاتيح البحث مرتبة ترتيباً مفروزاً (Sorted Order).
- الأدلة المقطعة (Hash Indices): مفاتيح البحث موزعة باستخدام تابع تقطيع (Hash Function).

تقييم فعالية الفهرس

عند اختيار نوع الفهرس، نقيمه بناءً على:

1. نوع الوصول الذي يدعمه بفعالية:
 - البحث عن قيمة محددة.
 - البحث ضمن مجال من القيم.
2. كلفة العمليات الأساسية (بحث، إضافة، حذف).
3. حجم التخزين الإضافي الذي يحتاجه الفهرس.

الأدلة المرتبة (Ordered Indices)

في الدليل المرتب (Ordered Index) يتم تخزين مداخل الفهرس (Index Entries) مرتبة ترتيباً مفروزاً (Sorted) حسب قيمة مفتاح البحث (Search Key Value). أي أن الفهرس نفسه يكون ملفاً مرتباً، مما يسمح بالبحث السريع.

أنواع الأدلة المرتبة:

1. الفهرس الرئيسي (Primary Index):

فهرس يُعرّف على ملف مرتب تسلسلياً (Sequentially Ordered File)، ويكون مفتاح البحث فيه هو نفسه الذي يحدد الترتيب الفيزيائي للسجلات داخل الملف. لذلك:

- يُسمى أيضاً (Clustering Index) الفهرس المُجمّع.
- لأنه يحدد كيف تتجمّع (cluster) السجلات داخل الكتل.



ملاحظة أرييسيزية:

مفتاح البحث في الفهرس الرئيسي:

- غالباً يكون المفتاح الرئيسي (Primary Key).
- لكن ليس شرطاً.

2. الفهرس الثانوي (Secondary Index):

فهرس يكون مفتاح البحث فيه **مختلفاً عن ترتيب الملف الأساسي**.
لذلك:

- يُسمى أيضاً Non-Clustering Index.
- لا يحدد الترتيب الفيزيائي للسجلات في الملف.

لماذا نحتاجه؟

لأننا كثيراً نحتاج البحث حسب واصفات ليست هي أساس الترتيب.

مثال:

الملف مرتب حسب account-number

ونريد البحث حسب:

- branch-name
- balance

الحل: إنشاء Secondary Index على هذه الحقول.

ملف تسلسلي مفهرس (Index-Sequential File)

ملف مرتب تسلسلياً + فهرس رئيسي (Primary Key).

أي:

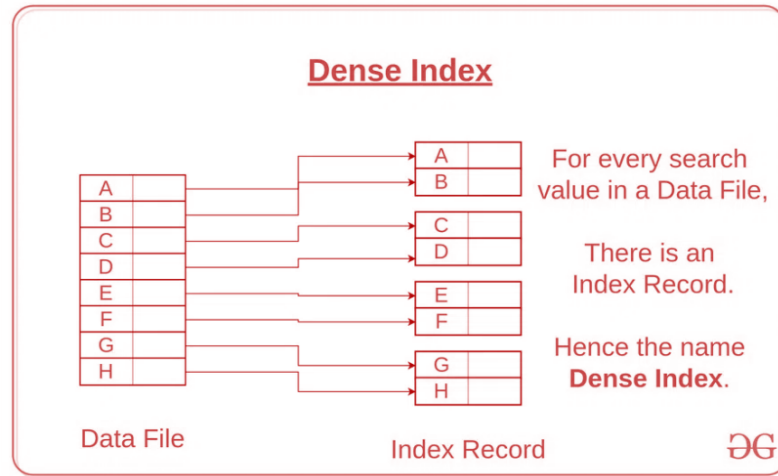
- الملف مرتب.
- والفهرس يشير إلى أماكن السجلات أو الكتل.
- يجمع بين مزايا التنظيم التسلسلي والفهرسة.

لا تخف من الطويل

خف من الوقوف



ملفات الفهرس الكثيف (Dense Index Files)

**الفهرس الكثيف (Dense Index):**

يحتوي مدخل فهرس (Index Record) لكل قيمة مفتاح بحث موجودة في الملف.
أي: لكل سجل في الملف يوجد مدخل في الفهرس.

المزايا:

- بحث سريع جداً.
- وصول مباشر للتسجيلة.

العيوب:

- يحتاج مساحة تخزين أكبر.
- كلفة أعلى عند الإضافة والحذف.

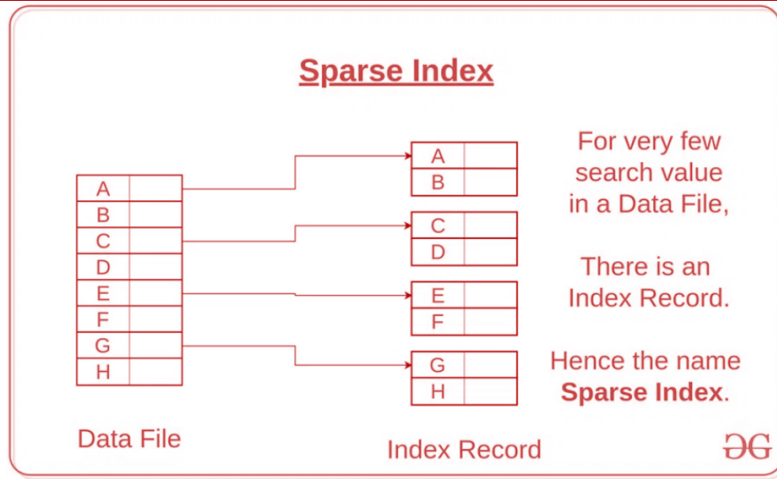


Brighton		A-217	Brighton	750	
Downtown		A-101	Downtown	500	
Mianus		A-110	Downtown	600	
Perryridge		A-215	Mianus	700	
Redwood		A-102	Perryridge	400	
Round Hill		A-201	Perryridge	900	
		A-218	Perryridge	700	
		A-222	Redwood	700	
		A-305	Round Hill	350	

الحياة لا تعطيك ما تريد

بل ما تعمل لأجله

ملفات الفهرس الأجوف (Sparse Index Files)

**الفهرس الأجوف (Sparse Index):**

يحتوي مداخل فهرسة لبعض قيم مفتاح البحث فقط، وليس لجميعها.

⚡ شرط أساسي:

لا يمكن استخدامه إلا إذا كان:

- الملف الأساسي مرتباً تسلسلياً حسب مفتاح البحث.

⚡ مميزاته:

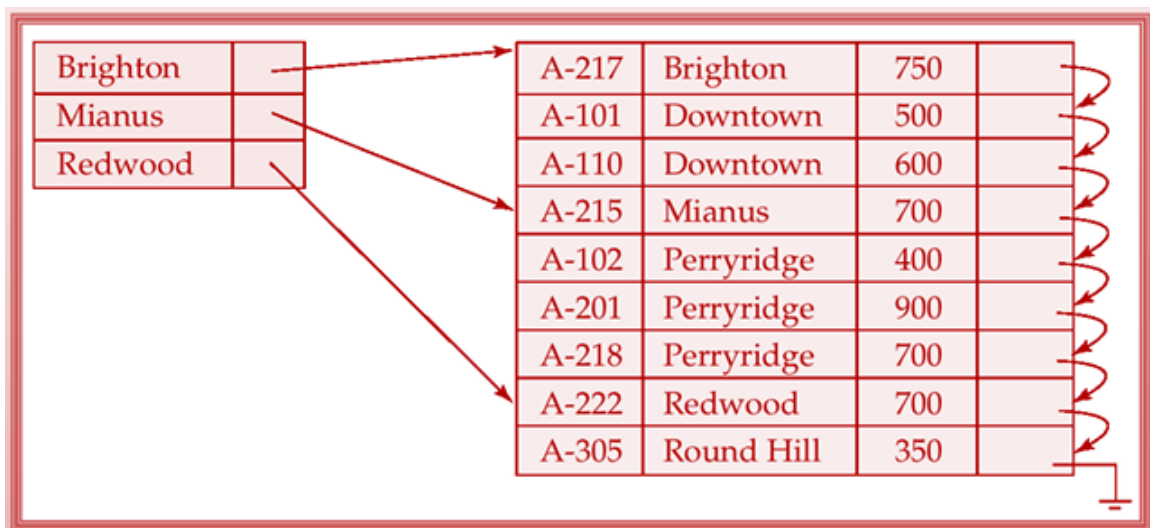
- يحتاج مساحة تخزين أقل.
- عدد التعديلات عليه أقل عند الإضافة والحذف.

⚡ عيبه:

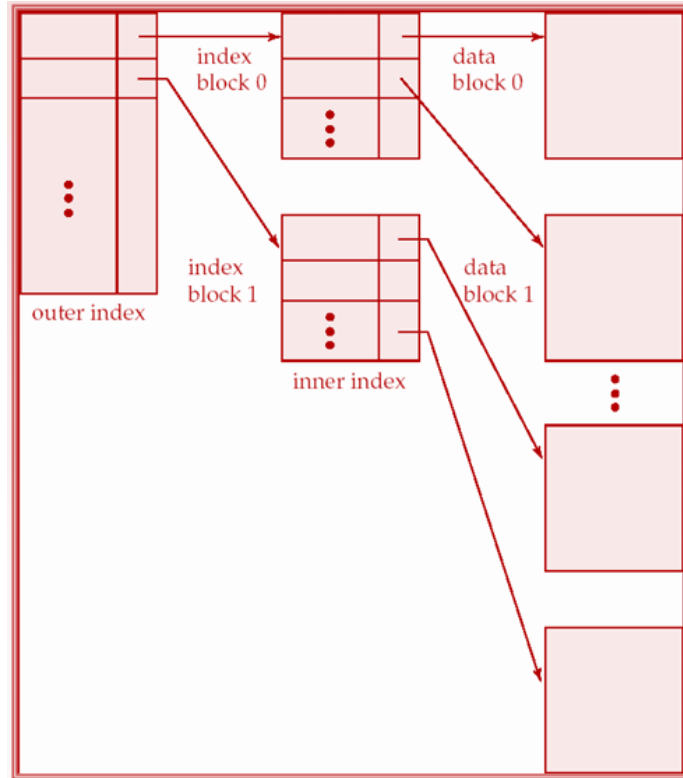
- أبطأ من الفهرس الكثيف عند البحث عن تسجيلة محددة.

مثال:

يوضح المثال فهرساً أجوفاً مبنياً على branch-name، حيث لا توجد مدخلة فهرس لكل سجل، بل فقط لبعض القيم التي تشير إلى بداية كتل في الملف الأساسي:



الفهرس متعدد المستويات (Multilevel Index)



عندما يكبر حجم الفهرس ولا يمكن تحميله كاملاً في الذاكرة الرئيسية، يصبح الوصول مكلفاً. الحل هو بناء فهرس على الفهرس نفسه.

- الفهرس الداخلي (Inner Index): هو الفهرس الرئيسي.
 - الفهرس الخارجي (Outer Index): فهرس أجوف مبني على الفهرس الداخلي.
- إذا بقي الفهرس الخارجي كبيراً، تُنشئ مستوى ثالث، وهكذا.

عند الإضافة أو الحذف، يجب تعديل جميع مستويات الفهرس.

تعديل الفهرس

1. تحديث (الفهرس عند الحذف):

الحالة العامة:

عند حذف تسجيلية (Record) من الملف الأساسي، يجب الانتباه إلى أن الفهرس يمثل قيم مفاتيح البحث (Search Key Values) الموجودة فعلياً في الملف.

القاعدة العامة:

إذا كانت التسجيلية المحذوفة:

هي التسجيلية الوحيدة التي تملك قيمة مفتاح البحث ← يجب حذف قيمة مفتاح البحث من الفهرس أيضاً.

لا في فهرس بمستوى واحد (Single-Level Index):

1. Dense Index:

في الفهرس الكثيف (Dense Index) يوجد مدخل فهرس لكل قيمة مفتاح بحث، لذلك يكون التحديث مباشرًا.

مثال:

لدينا الملف مرتب حسب account-number:

account-number
101
103
105
108

والفهرس الكثيف هو:

Search key	Pointer
101	-> rec1
103	-> rec2
105	-> rec3
108	-> rec4

عند حذف التسجيل ذات مفتاح البحث: 105

لا توجد تسجيلة أخرى بنفس القيمة ← نحذف مدخل 105 من الفهرس.

الحذف في Dense Index مشابه تماماً لحذف سجل من الملف الأساسي.

2. Sparse Index:

في الفهرس الأجوف (Sparse Index) لا يوجد مدخل لكل قيمة مفتاح بحث، بل فقط لبعض القيم (عادة أول قيمة في كل كتلة).

مثال:

نفترض أن الملف مقسم إلى كتل:

Block 1: 101, 103

Block 2: 105, 108

والفهرس الأجوف:

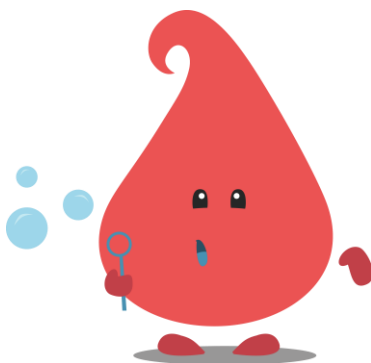
Search Key	Pointer
101	-> Block1
105	-> Block2

1. حذف تسجيلة لها مدخل في الفهرس:

نحذف التسجيل ذات القيمة 101 (أول سجل في Block 1)

بما أن 101 موجودة في الفهرس:

■ نستبدلها بالقيمة التالية في ترتيب المفتاح داخل نفس الكتلة وهي 103



Search Key	Pointer
103	-> Block1
105	-> Block2

2. القيمة التالية موجودة أصلاً في الفهرس:

إذا كانت القيمة التالية (مثلاً 103) موجودة أصلاً كمدخل في الفهرس:

▪ نحذف مدخل 101.

▪ دون استبدال.

هذه المعالجة تمنع التكرار داخل الفهرس.

2. تحديث الفهرس عند الإضافة:

عند إدخال تسجيلية جديدة:

نقوم بإجراء بحث (Lookup) عن قيمة مفتاح البحث الخاصة بالتسجيلية.

🔗 في فهرس بمستوى واحد (Single-Level Index):

1. Dense Index:

إذا لم تكن قيمة مفتاح البحث موجودة في الفهرس ← نضيف مدخل فهرس جديد (Index Entry).

إذا في Dense Index كل إضافة تسجيلية جديدة تقريباً تعني إضافة مدخل فهرس جديد.

2. Sparse Index:

إذا كان الفهرس يحتوي مدخلاً لكل كتلة (Block):

▪ لا نعدل الفهرس عند الإضافة داخل كتلة موجودة.

مثال 1: إضافة دون إنشاء كتلة جديدة

نضيف التسجيلية 104 داخل Block 2:

• لم تتغير بنية الكتلة.

• أول قيمة في الكتلة لم تتغير.

← لا تعديل على الفهرس.

من لا يحسن إدارة يومه

لن يحسن إدارة مستقبله

مثال 2: إضافة مع إنشاء كتلة جديدة

نفترض أن Block 2 مفقودة، فتم إنشاء:

• Block 3: 110, 115

← هنا نُضيف إلى الفهرس:

• أول قيمة مفتاح بحث في الكتلة الجديدة (110).

← يصبح الفهرس:

Search Key	Pointer
101	-> Block1
105	-> Block2
110	-> Block3

↪ بالنسبة Multilevel Index الفهرس متعدد المستويات:

- يتم تطبيق نفس مبدأ الإضافة والحذف.
- لكن التعديل لا يقتصر على مستوى واحد بل يتم على جميع مستويات الفهرس (Inner & Outer Index).

أي تغيير في المستوى الأدنى يجب أن ينعكس تدريجياً على المستويات الأعلى.

الأدلة الثانوية**↪ لماذا نستخدمها؟**

لأنها غالباً تحتاج:

- إيجاد جميع السجلات التي تحقق شرطاً على حقل ليس مفتاح البحث للفهرس الرئيسي.

أمثلة:

- إيجاد جميع الحسابات في فرع معين.
- إيجاد الحسابات ذات رصيد محدد أو ضمن مجال.

البنية:

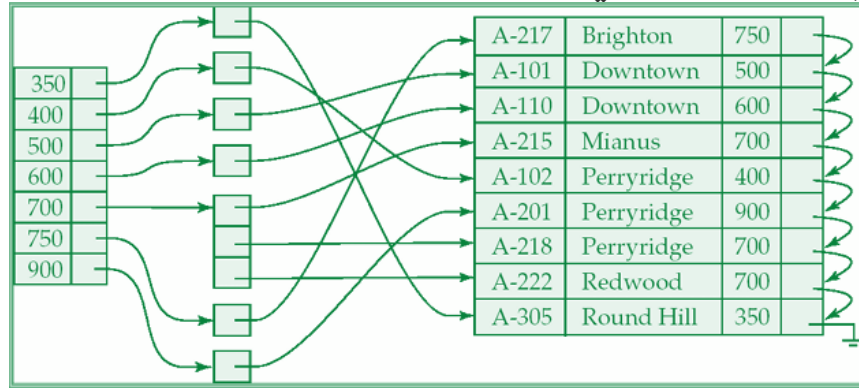
لكل قيمة مفتاح بحث:

- Index Entry يشير إلى حزم (Bucket) تحوي مؤشرات إلى جميع السجلات المطابقة.

ملاحظة مهمة:

الفهرس الثانوي يجب أن يكون Dense.

مثال: فهرس ثانوي على حقل الرصيد في جدول الحسابات



Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.

مقارنة بين الفهرس الأساسي والفهرس الثانوي

Primary and Secondary Indices

الفائدة الرئيسية للفهارس Indices:

- تُعطي تحسناً كبيراً في سرعة البحث (Search Performance) عن السجلات داخل الملفات الكبيرة، لأننا لا نضطر لقراءة الملف كاملاً.

ولكن:

هذه الفائدة لها ثمن (Cost).

كلفة تحديث الفهارس (Index Update Overhead)

عند أي تعديل على الملف الأساسي:

- Insertion
- Deletion
- Update

يجب على نظام إدارة قاعدة البيانات (DBMS) أن:

- يُحدّث كل Index معرف على هذا الملف

سواء كان:

- Primary Index
- Secondary Index

وهذا ما يسمى:

- Index Update Overhead

أي عبء إضافي على عمليات التعديل، وليس على البحث.



المسح التسلسلي (Sequential Scan)

1. باستخدام Primary Index:

لماذا هو فعال؟

- ترتيب السجلات في الملف الأساسي متوافق مع ترتيب مفتاح البحث.
- السجلات المطلوبة تكون غالباً:
- 1. في نفس Block.
- 2. أو في Blocks متجاورة.

عند تنفيذ Sequential Scan:

- يتم جلب Block.
 - ثم الذي يليه مباشرة.
 - دون قفز عشوائي على القرص.
- وهذا يسمى:
- Sequential Disk Access
- وهو الأسرع على القرص.



2. باستخدام Secondary Index:

لماذا هو مكلف؟

- ترتيب الفهرس لا يتطابق مع ترتيب السجلات في الملف.
- المؤشرات في الفهرس قد تشير إلى: Records موزعة على Blocks مختلفة تماماً.

أثناء المسح التسلسلي:

- كل Record قد يكون في Block مختلف.
- كل Block يحتاج عملية قراءة مستقلة من القرص.

وهذا يؤدي إلى:

- Random Disk Access
- وهو بطيء جداً.

Sequential Scan باستخدام Primary Index فعال لأن السجلات تُقرأ بترتيب متجاور من القرص، بينما Sequential Scan باستخدام Secondary Index مكلف جداً لأن كل سجل قد يتطلب قراءة Block مختلف من القرص، مما يضاعف زمن التنفيذ بشكل كبير.

من يهرب من المواجهة

يعيش عمره يبرر لا ينجز

الفرق بين الفهارس التسلسلية والفهارس متعددة المستويات

1. مساوئ استخدام الملفات المفهرسة تسلسلياً (Index-Sequential Files):

في الملفات المفهرسة تسلسلياً:

- الملف الأساسي مرتب.
 - يوجد Primary Index يشير إلى بدايات الكتل.
- لكن مع الزمن ومع كثرة العمليات، تظهر مشاكل مهمة:

1. مشكلة Overflow Blocks:

عند كثرة عمليات Insertion:

- لا يكون هناك مكان دائماً داخل الكتلة المناسبة.
 - فيتم إنشاء Overflow Blocks.
- مع ازدياد حجم الملف:
- يزداد عدد ال Overflow Blocks.
 - تصبح السجلات مرتبطة بسلاسل طويلة.

الأداء يتناقص كلما كبر حجم الملف، بسبب كثرة القفز بين الكتل الإضافية.

2. الحاجة إلى إعادة تنظيم الملف (Reorganization):

بسبب تراكم ال Overflow Blocks:

- يصبح الملف غير فعال.

ننظر إلى:

إعادة تنظيم الملف كاملاً بشكل دوري.
وهذه العملية:

- مكلفة زمنياً.
- تؤثر على توفر النظام.

2. محاسن استخدام الفهرس متعدد المستويات (Multilevel Index):

الفهرس متعدد المستويات لا يعتمد على ملف مرتب مع Overflow Blocks، بل يستخدم بنية هرمية من الفهارس.

1. إعادة تنظيم تلقائية للفهرس:

عند:

- الإضافة (Insertion).
- الحذف (Deletion).

يتم:

إعادة تنظيم الفهرس بشكل آلي وبدون الحاجة لإعادة بناء الملف الأساسي.

2. تغييران بسيطة ومحلية:

عمليات التعديل:

- لا تؤثر على كامل الفهرس.
 - بل تكون Local Changes:
 - 1. في عقد محددة.
 - 2. وعلى مستويات محددة.
- وهذا ما سنراه بوضوح لاحقاً في:
- B+Tree Index

3. لا حاجة لإعادة تنظيم الملف:من أهم الميزات:

لا نحتاج إلى إعادة تنظيم الملف الأساسي.

لأن:

- الفهرس نفسه يتكيف مع الإضافات والحذف.
- دون تراكم مشاكل مثل Overflow Blocks.

3. مساوئ استخدام الفهرس متعدد المستويات:

رغم ميزاته، له كلفة:

1. عمليات إضافية:

- كل إضافة أو حذف في الملف قد تتطلب:
- ← تعديلات إضافية في الفهرس وربما في أكثر من مستوى.

2. تخزين إضافي:

الفهرس متعدد المستويات يحتاج:

- مساحة تخزين أكبر.
- مقارنة بفهرس بسيط.

**ال فشل ليس في السقوط****بل في الاعتماد عليه**

رغم أن الفهرس متعدد المستويات يتطلب عمليات إضافة وحذف إضافية ومساحة تخزين أكبر، إلا أن محاسنه أقوى بكثير من مساوئه، ولذلك يُستخدم هذا النوع من الفهارس على نطاق واسع في أنظمة قواعد البيانات.

وهذا هو السبب الرئيسي لاستخدام:

- B+Tree

- B-Tree

في الأنظمة الحقيقية.

B+-Tree Index Files

B+-Tree هو:

- بديل مباشر ل Indexed-Sequential Files.
- مبني أساساً ك Multilevel Index لكن بطريقة منظمة ومضبوطة رياضياً.

لماذا نحتاج B+-Tree؟

عيوب Indexed-Sequential Files

مع ازدياد حجم الملف:

- تزداد Overflow Blocks.
- يقل الأداء.
- نحتاج إلى إعادة تنظيم الملف بالكامل.

وهذا غير مقبول في الأنظمة الكبيرة.

مزايا B+-Tree:

يعالج هذه المشكلة جزئياً:

1. يعيد تنظيم نفسه تلقائياً.
2. التعديلات تكون صغيرة ومحلية (Local Changes).
3. لا حاجة لإعادة تنظيم الملف كاملاً.
4. الأداء يبقى ثابتاً تقريباً مع نمو البيانات.

هذا هو السبب الأساسي لاستخدامه الواسع.

العيوب:

- كلفة إضافية عند Insertion / Deletion.
- مساحة تخزين إضافية.

Advantages of B+-trees outweigh disadvantages

مزاي B+-Tree تغلب مساوئه

ولهذا:

B+-trees are used extensively

خصائص B+-Tree الأساسية

1. شجرة متوازنة (Balanced Tree):

- جميع المسارات من Root إلى Leaf لها نفس الطول
- أي لا يوجد مسار أطول من الآخر.

وهذا يضمن:

- زمن بحث ثابت تقريباً.

2. عدد الأبناء (Fan-out):

كل عقدة غير ورقية وغير الجذر:

- لديها بين $(n/2)$ و n أبناء.

العقدة الورقية:

- تحتوي بين $(n/2)$ و $n-1$ قيم.

حالات خاصة:

الجذر:

- إذا لم يكن Leaf لديه على الأقل طفلان.

- إذا كان Leaf يمكن أن يحتوي من 0 إلى $n-1$ قيم.



بنية عقدة B+-Tree

■ Typical node

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

- K_i : Search-Key Values
- P_i : Pointers to children (for non-leaf nodes) or pointers to records or buckets of records

ال Search Keys داخل العقدة:

- $K_1 < K_2 < K_3 < \dots < K_{n-1}$

B+-Tree Leaf Nodes

العقد الورقية هي أهم جزء

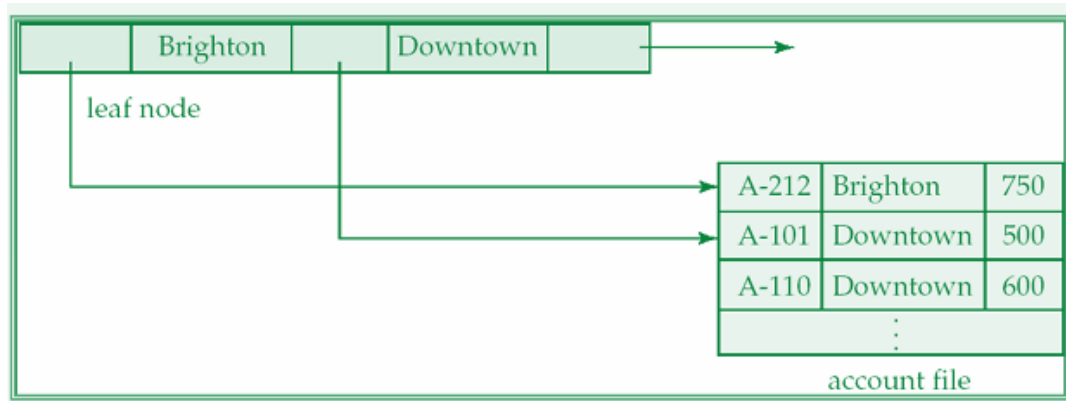
تحتوي:

- (Search-Key Value, Pointer).
- المؤشر قد يشير إلى:
 1. Record.
 2. أو Bucket إذا المفتاح ليس (Primary Key).

خصائص مهمة:

- القيم مرتبة.
- العقد الورقية مرتبطة ببعضها عبر Pointer إضافي ← يسمح ب Sequential Access سريع.

هذا سبب تفوق B+-Tree في range queries.



Non-Leaf Nodes

العقد غير الورقية:

- لا تحتوي بيانات فعلية.
- تشكل Multilevel Sparse Index فوق العقد الورقية.

وظيفتها:

توجيه البحث فقط

العقد الداخلية (Non-Leaf Nodes) تحقق الخواص التالية:

1. الخاصية الأولى:

جميع قيم Search-Key الموجودة في الشجرة الفرعية التي يشير إليها المؤشر P1 تكون أصغر من K1.

النجاح لا يحتاج سرعة، بل استمرارية

$$P_1 \mid 20 \mid P_2 \mid 50 \mid P_3$$

إذا كانت العقدة تحتوي:

فإن: جميع القيم الموجودة تحت P_1 أصغر من 20.

مثل: 18, 10, 5

2. الخاصية الثانية:

لكل مؤشر P_i حيث $1 \leq i \leq n-2$ ، تكون جميع قيم Search-Key في الشجرة الفرعية التي يشير إليها P_i أكبر أو تساوي K_{i-1} أو أصغر من K_i .

$$P_1 \mid 20 \mid P_2 \mid 50 \mid P_3$$

في نفس العقدة:

القيم تحت P_2 تحقق:

$$20 \leq \text{key} < 50$$

مثل: 45, 30, 20

3. الخاصية الثالثة:

جميع قيم Search-Key الموجودة في الشجرة الفرعية التي يشير إليها المؤشر الأخير P_n تكون أكبر أو تساوي K_{n-1} .

$$P_1 \mid 20 \mid P_2 \mid 50 \mid P_3$$

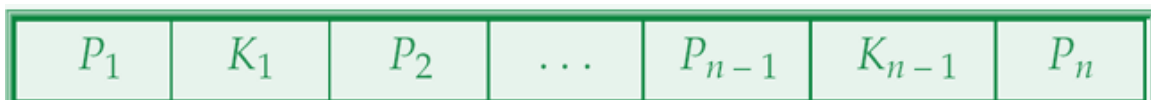
في نفس المثال:

تحت P_3 أكبر أو تساوي 50

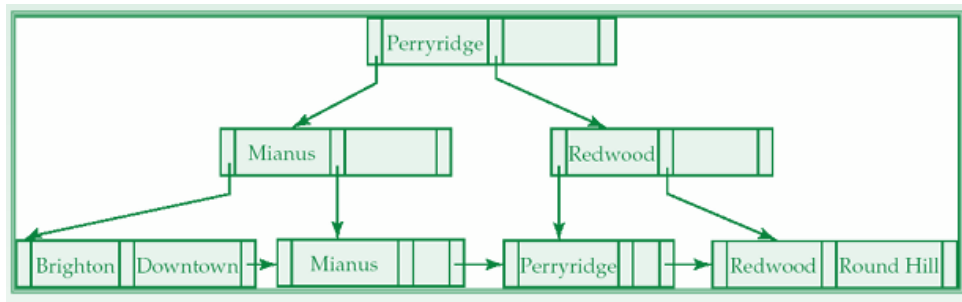
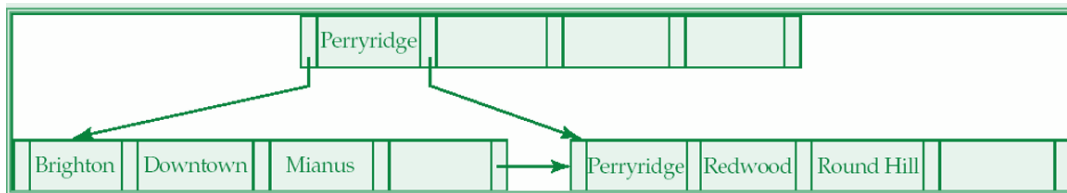
مثل: 80, 60, 50

هذه الخواص هي التي تسمح ل B+-Tree بأن:

- يوجه البحث في مسار واحد فقط.
- وضمن أن جميع الأوراق (Leaf nodes) على نفس العمق.
- ويحقق زمن بحث لوغاريتمي.



من اعتاد الأعداء، اعتاد الفشل

Examples of a B+-Tree:B+-Tree for account file ($n = 3$):B+-Tree for account file ($n = 5$):

توضيح اريسييزي للمثال

ماذا يعني $n = 5$ ؟

- n هو الحد الأعلى لعدد Pointers (children) في العقدة غير الورقية.
- أي أن العقدة الداخلية يمكن أن تحتوي حتى 5 أبناء.

من أجل $n = 5$:

1. Leaf Nodes:

القاعدة:

Leaf nodes must have between 2 and 4 value

لماذا 2 و 4؟

- الحد الأعلى:
- Leaf node يمكن أن تحتوي حتى $n - 1 = 4$ Search-Key Values
- الحد الأدنى:
- $\lceil (n - 1) / 2 \rceil = \lceil 4 / 2 \rceil = 2$

كل Leaf node يجب أن تحتوي على الأقل 2 وبحد أقصى 4 قيم.

مثال توضيحي:

إذا كانت Leaf node تحتوي: [101 | 103 | 105] ← صحيحة

أما [101] ← غير مسموحة وتسمى عندها (Underfull)

2. Non-leaf Nodes (غير الجذر):

القاعدة:

Non-leaf nodes other than root must have between 3 and 5 children

لماذا 3 و 5؟

- الحد الأعلى:
حتى $n = 5$ children
- الحد الأدنى:
 $\lceil n/2 \rceil = \lceil 5/2 \rceil = 3$

أي عقدة داخلية (غير الجذر) يجب أن تحتوي:

- 3 على الأقل.
- 5 على الأكثر من الأبناء.

مثال:

عقدة داخلية تحتوي:

2 children ✗

3 children ✓

5 children ✓

3. Root Node:

القاعدة الخاصة بالجذر:

Root must have at least 2 children

لماذا استثناء؟

- الجذر لا يلتزم بقاعدة $\lceil n/2 \rceil$.
- الهدف:
 1. السماح ببدء الشجرة.
 2. السماح بتقلصها بعد عمليات Deletion.
- الجذر:
 - يمكن أن يحتوي: طفلين أو أكثر.



الفرح الصغير يصنع يوماً كبيراً

■ وإذا أصبح لديه طفل واحد فقط:

1. يُحذف
2. يصبح ابنه هو الجذر الجديد (إذا أصبح الجذر يحتوي مؤشراً واحداً فقط، فهذا يعني أنه لا يضيف أي توجيه، لذلك يُحذف، ويرفع ابنه الوحيد ليصبح هو الجذر الجديد للشجرة).

مقارنة بين المثالين:

الخاصية	n = 3	n = 5
القيم القصوى في Leaf	4	2
أقل قيم في Leaf	2	1
ارتفاع الشجرة	أصغر	أكبر
عدد العقد	أقل	أكثر
كفاءة البحث	أعلى	أقل

من خلال المثالين $n = 3$ و $n = 5$ نلاحظ أن زيادة عدد المؤشرات في العقدة تؤدي إلى شجرة أعرض وأقصر، مما يقلل عدد المستويات وعدد عمليات الوصول للقرص أثناء البحث.

نهاية التوضيح الاربيسيزي

ملاحظات حول B+-Tree:

- عدد مستويات الشجرة قليل نسبياً بسبب عدد الأبناء (children / pointers) الذي يمكن أن تمتلكه العقدة الواحدة في الشجرة (fan-out) الكبير.
- عدد المستويات يعتمد على حجم الملف الأساسي (عدد القيم).
- عمليات Insertion و Deletion تتم بتعديلات محلية وفي وقت لوغاريتمي.
- لا حاجة إلى إعادة تنظيم كامل للملف.
- عمليات Search فعالة وتتم بزمان Logarithmic.

الاستعلام باستخدام فهرس B+-Tree

1. استخدام B+-Tree في البحث (Search using B+-Tree):

الهدف:

إيجاد جميع السجلات التي تملك قيمة محددة لمفتاح البحث k.

خطوات البحث (بشكل منطقي):

1. نبدأ من عقدة الجذر (Root).
2. داخل كل عقدة داخلية:
 - نبحث عن أصغر قيمة K_i حيث: $K < K_i$
 - إذا وجدناها ← نتابع عبر المؤشر P_i .

- إذا كانت $K \geq K_{n-1}$ ← نتابع عبر المؤشر P_n .
- 3. نكرر الخطوات السابقة طالما العقدة ليست Leaf.
- 4. عند الوصول إلى Leaf Node:
- إذا وجدنا $K_i = K$ ← نتبع المؤشر للوصول إلى التسجيلة أو ال (bucket).
- إذا لم نجدها ← التسجيلة غير موجودة.

البحث يتم دائماً عبر مسار واحد من الجذر إلى ورقة واحدة فقط.

2. كفاءة البحث في B+-Tree:

عدد العقد التي نزورها أثناء البحث أقصى عدد من العقد التي نمرّ عليها: $[\log [n/2] (k)]$

حيث:

■ K = عدد قيم Search-Key.

■ n = fan-out

مثال عملي:

حجم العقدة = حجم Block القرص $\approx 4 \text{ KB}$

حجم Index Entry $\approx 40 \text{ bytes}$

إذاً: $n = 100$

إذا كان: $k = 1,000,000$

أي نحتاج:

الوصول إلى 4 عقد فقط لإيجاد أي سجل

وهذا ممتاز لأن:

كل عقدة $\approx$ عملية Disk I/O واحدة.

3. تنظيم الملف باستخدام B+-Tree (B+-Tree File Organization):

بدل أن تكون B+-Tree مجرد Index نستخدمها لتنظيم الملف نفسه.

كيف؟

- Leaf nodes:
- 1. تخزن السجلات مباشرة.
- 2. وليس فقط مؤشرات.
- Non-leaf nodes:
- 1. تبقى فهرساً للتوجيه فقط.

مميزات هذا الأسلوب:

- لا يوجد ملف بيانات منفصل.
- لا توجد Overflow Blocks.
- Leaf nodes تبقى نصف ممتلئة على الأقل (يعني أن كل عقدة في B+-Tree (عدا الجذر) يجب ألا يقل عدد القيم أو المؤشرات فيها عن الحد الأدنى المسموح، أي لا تقل عن نصف سعتها تقريبًا).
- عمليات Insertion / Deletion: على الشجرة وعلى الملف تتم بنفس الآلية وببنفس الكفاءة.

باستخدام B+-Tree File Organization، تُخزن السجلات مباشرة في الأوراق.

B-Tree Index Files

الفكرة الأساسية:

- B-Tree مشابه في البنية العامة ل B+-Tree.
- لكن مع فرق جوهري:
- Search-key value تظهر مرة واحدة فقط في الشجرة.

ما معنى ذلك؟

في B-Tree:

- إذا ظهرت قيمة مفتاح بحث في Non-leaf node ← فإنها لا تظهر مرة أخرى في Leaf node.

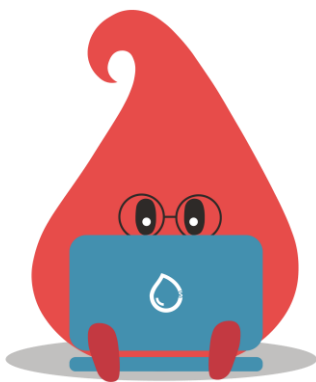
بعكس B+-Tree التي فيها:

- Search-Key values تظهر في:
- 1. Non-leaf nodes للتوجيه.
- 2. Leaf nodes للوصول إلى البيانات.

نتيجة هذا الاختلاف:

بما أن Search-key لا يتكرر:

- يجب أن تحتوي Non-leaf nodes في B-Tree على:
- Pointer إضافي مع كل Search-key.
- هذا المؤشر يشير مباشرة إلى:
- 1. Record.
- 2. أو Bucket.



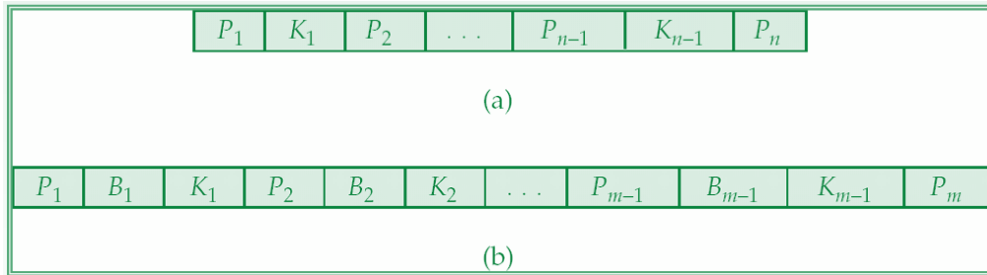
العقد الداخلية في B-Tree تشارك في البحث والوصول للبيانات.

من لم يحدد أولوياته، استهلكه كل شيء

مؤشرات Bi في B-Tree:

المؤشرات Bi في العقد الداخلية:

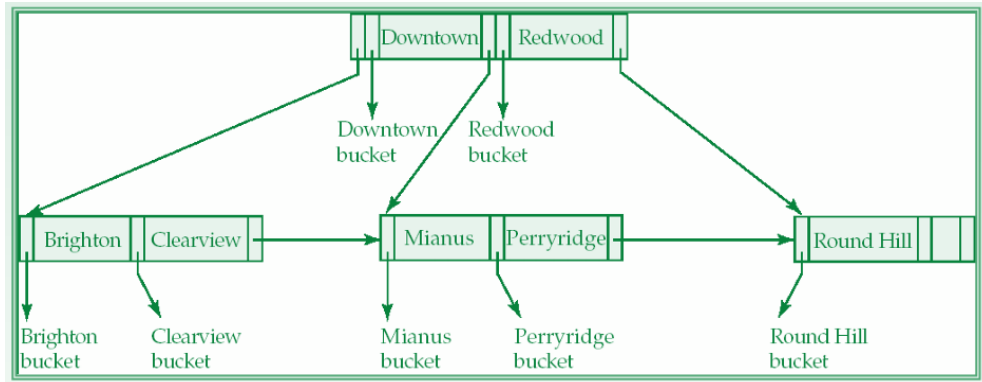
- قد تشير مباشرة إلى:
- 1. سجل في الملف الأساسي.
- 2. أو Bucket.
- وليس فقط إلى عقدة فرعية كما في B+-Tree.



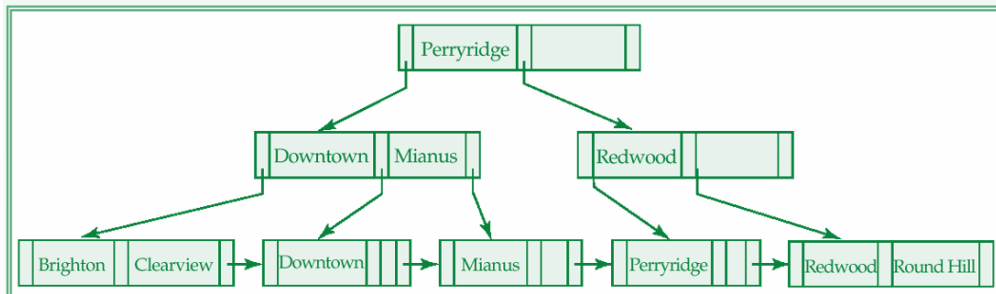
Nonleaf node – pointers Bi are the bucket or file record pointers.

مثال يوضح الفرق بين B-Tree و B+-Tree على نفس البيانات:

B-Tree: بعض القيم يتم العثور عليها قبل الوصول إلى Leaf.



B+-Tree: كل القيم الفعلية موجودة فقط في leaf nodes.



مزايا B-Tree:

- قد تحتاج عدد عقد أقل من B+-Tree.
- أحياناً يمكن إيجاد Search-key قبل الوصول إلى leaf node.

B-Tree مساوئ

- فقط عدد قليل من القيم يُعثر عليه مبكراً.
- Non-leaf nodes أكبر حجماً.
- fan-out أقل.
- عمق الشجرة أكبر.
- عمليات Deletion / Insertion أكثر تعقيداً.
- التنفيذ أصعب.

Typically, advantages of B-Trees do not outweigh disadvantages.

عادةً، مزايا B-Tree لا تغلب مساوئه.

لذلك تستخدم B+-Tree على نطاق أوسع في أنظمة قواعد البيانات.

Static Hashing (التقطيع الثابت)

مفهوم ال Bucket:

- Bucket هي وحدة تخزين تحتوي سجلاً واحداً أو أكثر.
- غالباً ما تكون ال Bucket مساوية ل Disk Block.
- أي أن كل Bucket تمثل موقعاً فعلياً على القرص.

Hash File Organization ↙

في Hash File Organization:

- يتم تحديد موقع التسجيل مباشرةً من: Search-Key Value
- وذلك باستخدام: Hash Function

أي أننا:

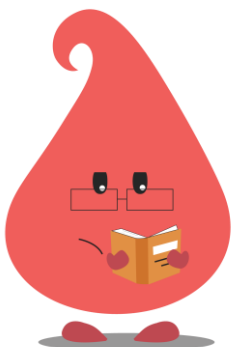
لا نبحث داخل الملف، بل نحسب العنوان مباشرة.

↘ في التقطيع الثابت (Static Hashing):

- نستخدم Hash Function.
- نطبّقها على Search-Key Value.
- الناتج يحدد: أي Bucket سيتم تخزين التسجيل فيها.
- عدد ال Buckets ثابت ولا يتغير.

لا تختصر الظروف المثالية

اصنع الممكن



لدينا ملف account، وعدد ال Buckets = 10

ونستخدم Hash Function التالية:

$$h(\text{name}) = (\sum \text{name}[i]) \bmod 10$$

→ The hash function returns the sum of the binary representations of the characters modulo 10.

النتائج:

$$h(\text{Perryridge}) = 5$$

$$h(\text{Round Hill}) = 3$$

$$h(\text{Brighton}) = 3$$

نلاحظ أن السجلات ذات القيم:

Brighton و Round Hill

تخزن في نفس ال Bucket.

وهذا يقود إلى مشكلة...

bucket 0				bucket 5	A-102	Perryridge	400
					A-201	Perryridge	900
					A-218	Perryridge	700
bucket 1				bucket 6			
bucket 2				bucket 7	A-215	Mianus	700
bucket 3	A-217	Brighton	750	bucket 8	A-101	Downtown	500
	A-305	Round Hill	350		A-110	Downtown	600
bucket 4	A-222	Redwood	700	bucket 9			

مشكلة التصادم (Collision):

يحصل عندما نعطي Hash Function نفس القيمة لأكثر من Search-Key.

أي:

أكثر من سجل يريد الدخول إلى نفس ال Bucket.

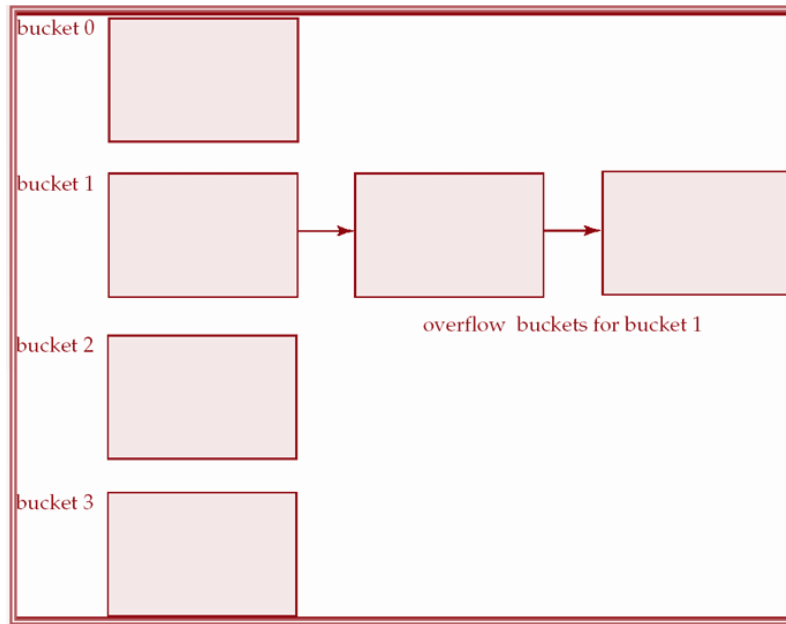
حل التصادم Overflow Chaining:

يستخدم:

Overflow Blocks

تربط على شكل Linked list

إن أردت نتيجة مختلفة، غيّر سلوك الآن



هذا الأسلوب يسمى:

- Closed Hashing (تقطيع مغلق)
- أو Overflow Chaining

ملاحظة:

يوجد أسلوب آخر: Open Hashing لكنه غير مناسب غالباً لتطبيقات قواعد البيانات.

Hash Indices (الفهارس المتقطعة)

ما هو Hash Index؟

- فهرس يستخدم Hashing
- لتنظيم:
 1. Search-Key Values
 2. Pointers إلى السجلات أو ال Buckets

استخدامات Hashing:

يمكن استخدام التقطيع في:

1. File Organization
2. Index Structures

ملاحظة مهمة:

Hash Index هو دائماً Secondary Index

لماذا؟ لأن الملف غير مرتب ولا يمكن استخدامه للمسح التسلسلي أو Range Queries.

من لم يبدأ صغيراً، لن يصل كبيراً

ملاحظة اصطلاحية:

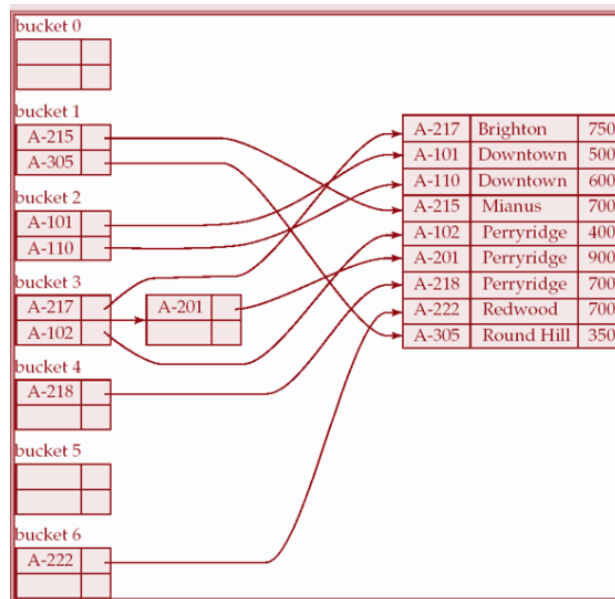
إذا كان الملف نفسه منظماً باستخدام Hashing:

- لا حاجة لبناء Hash Index إضافي.
- نستخدم مصطلح: Hashed File

مع ذلك:

نستخدم مصطلح Hash Index للإشارة إلى:

- Hash Indices
- Hash Organized Files

Example of Hash Index:

توضيح لدالة أو تابع التقطيع (Hash Function):

- نرسم لها ب h
- وهي دالة:

$$h: K \rightarrow B$$

منطلقها K ومستقرها B حيث:

- K: مجموعة قيم Search-Key
- B: مجموعة عناوين ال Buckets

أنت أقوى مما تعتقد

لكنك لن تعرف حتى تتحرك

Deficiencies of Static Hashing

مساوئ التقطيع الثابت

المشكلة الجوهرية في Static Hashing:

- Hash Function تربط القيم بعدد ثابت من Buckets.
- ولكن:**

قواعد البيانات تنمو أو تتقلص مع الزمن.

الحالات المحتملة:

1. نمو قاعدة البيانات:

- عدد ال Buckets يصبح غير كافٍ
- تزداد Collisions
- تزداد Overflow Buckets
- ينخفض الأداء

2. تخصيص Buckets مسبقاً:

- يؤدي إلى:
 1. Buckets Underfull
 2. هدر كبير في المساحة

3. تقلص قاعدة البيانات:

- مساحة كبيرة تبقى غير مستخدمة.

الحل التقليدي:

- إعادة تنظيم الملف دورياً باستخدام Hash Function جديدة.

لكن هذا الحل:

مكلف ويعرقل العمل الطبيعي للنظام.

الحل الأفضل:

- السماح بتعديل عدد ال Buckets ديناميكياً.
- وهذا يقود إلى فكرة Dynamic Hashing.

ملاحظة ارييسيزية:

لم يذكر اسم الفكرة (Dynamic Hashing) في المحاضرة بشكل مباشر ولكن ذكرت فكرة اللجوء الى تقنية تسمح بتعديل عدد الرزم بشكل ديناميكي.

المقارنة بين Hashing و Ordered Indexing

عوامل الاختيار:

- اختيار نوع الفهرس يعتمد على:
- 3. كلفة إعادة التنظيم الدوري.
- 4. تكرار عمليات Insertion و Deletion.
- 5. نوع الاستعلامات المتوقعة.

متى يكون Hashing أفضل؟

عند الاستعلام عن:

- قيمة محددة لمفتاح البحث
WHERE key == value

هنا استخدام Hashing ممتاز

متى يكون Ordered Indexing أفضل؟

عند الاستعلام عن:

- مجال قيم (Range Queries)
WHERE key BETWEEN x AND y

هنا الأفضل استخدام Ordered Indexing



Index Definition in SQL

تعريف الفهرس بلغة SQL

في SQL يمكن إنشاء فهرس (Index) لتحسين سرعة الوصول إلى البيانات باستخدام الأمر:

```
create index <index-name> on <relation-name>
(<attribute-list>)
```

مثال:

```
create index b_index on branch (branch_name);
```

هذا الأمر:

- ينشئ فهرساً باسم b_index
- على الجدول branch
- باستخدام العمود branch_name كمفتاح بحث (Search Key)

الهدف من الفهرس:

- تسريع عمليات SELECT
- تقليل عدد عمليات القراءة من القرص

Unique Index ↩

create unique index <index-name> on <relation-name> (attribute);

Unique Index يضمن:

- عدم تكرار قيم مفتاح البحث.
- أي أن العمود المفهرس يمثل Candidate Key.

متى يستخدم؟

- عندما نريد التأكد أن القيم فريدة.
- مثل: رقم الهوية، رقم الحساب، البريد الإلكتروني

↩ لحذف فهرس:

drop index <index-name>;

Multiple-Key Access

الوصول باستخدام عدة مفاتيح

في بعض الاستعلامات نحتاج البحث باستخدام أكثر من شرط

مثال:

```
select account_number
from account
where branch_name = "Perryridge"
and balance = 1000 ;
```

هذا الاستعلام يحتوي شرطين:

- branch_name = "Perryridge"
- balance = 1000

↩ الحالة الأولى: فهارس منفصلة (Separate Indices):

نفترض وجود:

- Index على branch_name
- Index على balance

استراتيجية التنفيذ:

1. استخدام فهرس branch_name :
لإيجاد جميع الحسابات في فرع Perryridge
2. استخدام فهرس balance :
لإيجاد جميع الحسابات التي رصيدها 1000

3. إجراء تقاطع (Intersection) بين المجموعتين.
4. الناتج هو السجلات التي تحقق الشرطين معاً.

هذه الطريقة:

فعالة لكنها تحتاج عمليات إضافية (تقاطع نتائج).

Indices on Multiple Attributes

الفهارس المعتمدة على عدة أعمدة

Composite Index ↙

فهرس يُنشأ على أكثر من عمود معاً، مثل: (branch_name, balance).

مثال:

```
where branch_name = "Perryridge" and balance = 1000
```

عند وجود Composite Index على (branch_name, balance):

- يتم الوصول مباشرة إلى السجلات التي تحقق الشرطين
- دون المرور بمراحل وسيطة
- أكثر كفاءة من استخدام فهارس منفصلة

ملاحظة مهمة جداً (ترتيب الأعمدة):

الفهرس المركب يعتمد على ترتيب الأعمدة.

يعمل بكفاءة مع:

```
where branch_name = "Perryridge" and balance < 1000
```

لأن:

- الشرط الأول على العمود الأول في الفهرس والشرط الثاني تابع له ولا يعمل بكفاءة مع:

```
where branch_name < "Perryridge" and balance = 1000
```

السبب:

- الشرط الأول غير محدد بدقة فلا يمكن الانتقال مباشرة للعمود الثاني

هنا قد نحصل على:

سجلات تحقق شرطاً واحداً فقط ثم يتم التحقق من الشرط الثاني لاحقاً.

Grid Files (الملفات الشبكية)

Grid File: هي بنية فهرسة تستخدم لتسريع معالجة الاستعلامات التي تحتوي على أكثر من مفتاح بحث (multiple attributes)، خاصة الاستعلامات التي تعتمد على المقارنات.

الفكرة الأساسية:

- Grid File تعتمد على تقسيم فضاء القيم إلى شبكة (Grid).
- كل بعد في الشبكة يمثل صفة (Attributes) من صفات مفتاح البحث.
- كل خلية في الشبكة (Cell) تشير إلى Bucket يحتوي السجلات المطابقة.

عدد أبعاد الشبكة = عدد صفات مفتاح البحث.

مكونات Grid File

1. Grid Array (الشعاع الشبكي):

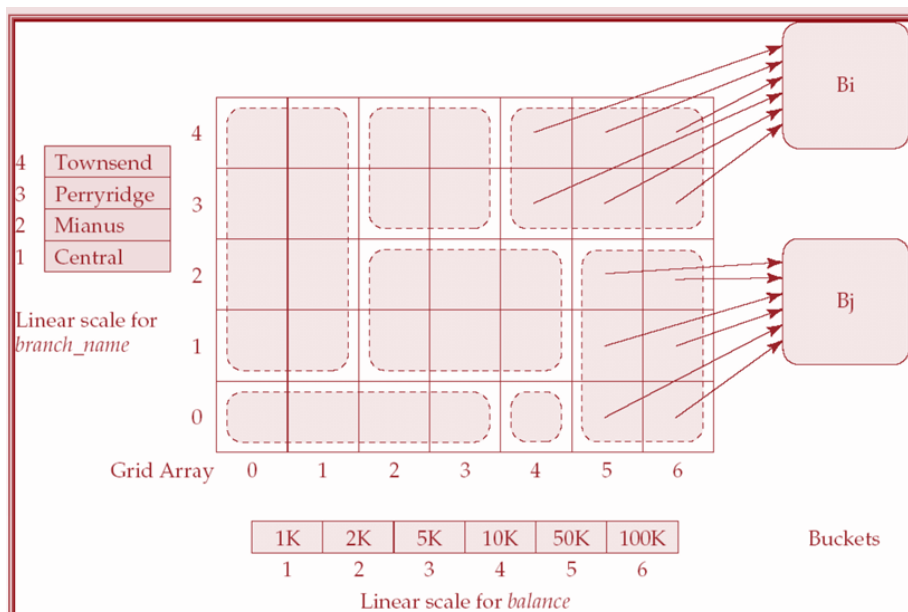
- مصفوفة متعددة الأبعاد.
- كل عنصر فيها يشير إلى Bucket.

2. Linear Scales تدرج خطي:

- لكل صفة يوجد تدرج خطي يقسم مجال قيمها.
- يستخدم لتحديد موقع القيمة داخل الشبكة.

3. Buckets:

- وحدات تخزين تحتوي السجلات.
- يمكن أن تشير عدة خلايا إلى نفس الـ bucket.



⚡ كيف يتم البحث؟

لإيجاد السجلات المطابقة لقيمة أو مدى معين:

1. نحدد موقع القيم على التدرج الخطي لكل صف.
2. نحدد الصف والعمود (أو الخلية) في ال Grid Array.
3. نصل إلى ال bucket المشار إليه.
4. نبحث داخل ال bucket عن السجلات المطلوبة.

⚡ الاستعلامات المدعومة بكفاءة:

Grid Files ممتازة للتعامل مع الاستعلامات من الشكل:

- $(a1 \leq A \leq a2)$
- $(b1 \leq B \leq b2)$
- $(a1 \leq A \leq a2 \text{ AND } b1 \leq B \leq b2)$

أي:

- شرط على صفة واحدة أو شروط متعددة معاً (AND).

مثال (Account File):

الصفات:

- branch-name
- balance

استعلام:

where branch-name < "Perryridge" and balance = 100k

الخطوات:

1. نحدد مجال branch-name على التدرج الخطي.
2. نحدد قيمة balance على تدرجها.
3. نحدد مجموعة الخلايا المطابقة.
4. نفحص ال buckets المرتبطة بهذه الخلايا.

⚡ الإضافة (Insertion):

عند إضافة سجل جديد:

- نحدد الخلية المناسبة.
- نضيف السجل إلى ال bucket.

في حال امتلاء ال bucket:

- يتم إنشاء bucket جديد.
 - يتم ربطه بالخلايا التي تشير إليه.
- الفكرة تشبه Dynamic Hashing لكن مع عدة أبعاد.

ملاحظات مهمة:

- اختيار التدرج الخطي يجب أن يكون مدروساً لضمان توزيع منتظم للسجلات.
- يمكن أن تتطلب البنية:
- إعادة تنظيم دورية.
 - مساحة تخزين إضافية كبيرة.
 - لكنها قوية جداً مع:
 - الاستعلامات متعددة الصفات.
 - Range queries.

Bitmap Indices

Bitmap Index هو نوع خاص من الفهارس صُمم لزيادة كفاءة الاستعلامات التي تعتمد على عدة صفات (Multiple Attributes)، خصوصاً عندما تكون هذه الصفات ذات عدد قليل من القيم.

الفكرة الأساسية:

- نفترض أن سجلات العلاقة مرقمة تسلسلياً (i).
- لكل قيمة ممكنة لصفة معينة يوجد Bitmap.
- ال Bitmap هو مصفوفة من البتات (Array of bits):
- 1. طولها = عدد السجلات.
- 2. كل بت يمثل سجلاً واحداً.

كيف يعمل Bitmap؟

لصفة معينة وقيمة معينة v:

- إذا كانت قيمة الصفة في السجل رقم i تساوي v ← البت رقم i = 1.
- غير ذلك ← البت = 0.



ليس ضعفاً أن تتعب

الضعف أن تتوقف

صفة gender:

القيم الممكنة {Male, Female}:

عدد السجلات = 6

• Bitmap لل Male:

1 0 1 1 0 0

• Bitmap لل Female:

0 1 0 0 1 1

لماذا Bitmap مفيد؟

- ممتاز للاستعلامات التي تحتوي:

1. AND

2. OR

3. NOT

- لأن العمليات تنفذ مباشرة على ال bits.

مثال على Bitmap والاستعلامات متعددة الصفات:

WHERE country = 'Syria' AND status = 'online'

الخطوات:

1. نأخذ bitmap للقيمة country = Syria

2. نأخذ bitmap للقيمة status = online

3. نطبق AND

4. البتات الناتجة = السجلات المطلوبة

الصفات المناسبة ل Bitmap:

Bitmap فعال جداً مع صفات مثل:

- النوع (ذكر أو أنثى).

- الحالة الاجتماعية.

- البلد.

- المستوى الوظيفي.

- مستويات الدخل (مقسمة إلى فئات مثل: 0-9999، 10000-19999، ...)

غير مناسب لصفات ذات قيم كثيرة جداً (مثل رقم الحساب).

الـ حجم والكفاءة:

- حجم Bitmap صغير جداً مقارنة بحجم العلاقة.
- العمليات تتم على مستوى: words 64-bit أو 32-bit
- لذلك التنفيذ سريع جداً (CPU-friendly).

مثال:

لدينا الجدول التالي ولدينا فهرسين bitmaps أحدهما من أجل الحقل gender والآخر من أجل الحقل income-level:

record number	name	gender	address	income-level	Bitmaps for gender		Bitmaps for income-level	
		m			m	10010	L1	10100
		f			f	01101	L2	01000
0	John	m	Perryridge	L1			L3	00001
1	Diana	f	Brooklyn	L2			L4	00010
2	Mary	f	Jonestown	L1			L5	00000
3	Peter	m	Brooklyn	L4				
4	Kathy	f	Perryridge	L3				

التعامل مع NULL:

- يجب وجود Bitmap خاص لقيم NULL.
- عند حساب: NOT (A = v)

يجب: مقاطعتها مع ExistenceBitmap وهو:

Bitmap يحدد أي السجلات لديها قيمة فعلية للصفة A ليست NULL:

(Not bitmap (A = v)) AND ExistenceBitmap

الحذف في Bitmap:

عند حذف سجل:

- يجب تحديث جميع ال bitmaps.
- الانتباه خصوصاً عند استخدام NOT.

Bitmap + B+-Tree

- يمكن استخدام Bitmap بدل Tuple-ID في أوراق B+-Tree.
- هذا الدمج:
- 1. يستفيد من سرعة Bitmap.
- 2. ومن تنظيم B+-Tree.
- مفيد عندما تظهر القيمة نفسها في عدد كبير من السجلات.

The End...